1. Online Learning Framework

1. Protocol

Sequential game (T rounds):

1. Learner chooses action/prediction
2. Environment reveals outcome
3. Learner observes loss and updates

Key difference from PAC:

- No distributional assumptions
- Adversarial environment allowed
- Performance measured by regret, not generalization error

2. Regret Definition

Regret: Difference between learner's cumulative loss and best fixed strategy in hindsight:

$$R_T = \sum_{t=1}^{T} \ell_t - \min_{i=1,\dots,n} \sum_{t=1}^{T} \ell_{t,i}$$

where ℓ_t is learner's loss at round t and $\ell_{t,i}$ is loss of expert i.

Goal: Sublinear regret $R_T = o(T)$ (average regret $\frac{R_T}{T} \to 0$).

Optimal rate: $O\left(\sqrt{T}\right)$ in adversarial setting.

3. Expert Advice Setting

Setup: n experts, each making predictions. Learner combines expert advice.

Two paradigms:

1. **Deterministic:** Always follow one expert
2. **Randomized:** Sample expert according to probability distribution

Key insight: Randomization enables better regret bounds.

2. Multiplicative Weights Update (MWU)

1. Algorithm

Initialize: $w_1(i) = 1$ for all experts $i = 1, \dots, n$

For round $t = 1, \dots, T$:

1. Define probability distribution:

$$p_t(i) = \frac{w_t(i)}{\sum_{j=1}^{n} w_t(j)}$$

2. Sample expert $i \sim p_t$ (or use expected loss)
3. Observe losses $\ell_{t,1}, \dots, \ell_{t,n}$
4. Update weights:

$$w_{t+1}(i) = w_t(i) \exp\left(-\eta \ell_{t,i}\right)$$

Parameter: Learning rate $\eta > 0$ (typically $\eta = \sqrt{\frac{\log n}{T}}$).

2. Intuition

Exponential reweighting: Experts with lower losses get exponentially higher weight.

Learning rate tradeoff:

- Large η: Fast adaptation, but unstable
- Small η: Stable, but slow to adapt
- Optimal: $\eta = \sqrt{\frac{\log n}{T}}$ balances both

Connection to AdaBoost: AdaBoost uses $D_{t+1}(i) \propto D_t(i) \exp(-\alpha_t y_i h_t(x_i))$, same exponential update!

3. Regret Bound

Theorem: MWU with $\eta = \sqrt{\frac{\log n}{T}}$ achieves:

$$R_T \leq 2\sqrt{T \log n}$$

assuming losses in $[0, 1]$.

Key features:

- $O\left(\sqrt{T}\right)$ regret (sublinear)
- Logarithmic in number of experts n
- No assumptions on loss sequence
- Works against adversarial environment

3. Regret Analysis

1. Potential Function Method

Key idea: Track total weight $\Phi_t = \sum_{i=1}^{n} w_t(i)$.

Step 1: Relate weight change to losses:

$$\frac{\Phi_{t+1}}{\Phi_t} = \sum_{i=1}^{n} p_t(i) \exp\left(-\eta \ell_{t,i}\right)$$

Step 2: Use inequality $\exp(-\eta x) \leq 1 - \eta x + \eta^2 x^2$ for $x \in [0, 1]$:

$$\frac{\Phi_{t+1}}{\Phi_t} \leq 1 - \eta \left(\sum_i p_t(i) \ell_{t,i}\right) + \eta^2 \left(\sum_i p_t(i) \ell_{t,i}^2\right)$$

Step 3: Telescope over T rounds and solve for regret.

2. Lower Bound

Theorem: For any online algorithm, there exists a loss sequence with:

$$R_T \geq \Omega\left(\sqrt{T}\right)$$

Implication: MWU is optimal (up to constants and log factors).

Proof sketch: Use adversarial construction where best expert is always opposite of learner's choice.

4. Weighted Majority Algorithm

1. Deterministic Version

Algorithm:

1. Maintain weights $w_t(i)$ for each expert
2. At each round, follow majority vote weighted by w_t
3. Halve weight of experts that make mistakes:

$$w_{t+1}(i) = \begin{cases} w_t \frac{i}{2} & \text{if expert } i \text{ wrong} \\ w_t(i) & \text{otherwise} \end{cases}$$

Mistake bound: At most $2.41(m^* + \log n)$ mistakes, where m^* is mistakes of best expert.

2. Randomized Version

Algorithm: Same as MWU with $\eta = \frac{1}{2}$ (corresponding to halving).

Regret: Expected loss at most $(1 + \varepsilon)m^* + \frac{\log n}{\varepsilon}$ for any $\varepsilon > 0$.

Advantage: Works when no single expert is highly accurate.

5. Online Convex Optimization Preview

1. From Discrete to Continuous

Discrete: n experts → probability distribution on simplex

Continuous: Infinite decision space $\mathcal{K} \subset \mathbb{R}^d$ → optimization in convex set

Key algorithms:

- Online Gradient Descent (OGD)
- Follow-The-Regularized-Leader (FTRL)

Same regret rate: $O(\sqrt{T})$ persists!

2. Connection to SGD

Stochastic Gradient Descent: Special case of online learning where losses are i.i.d. samples from a distribution.

Difference:

- SGD: Stochastic, i.i.d. gradients
- OGD: Adversarial, arbitrary sequence

Result: SGD analysis follows from OGD by replacing worst-case with expectation.

6. Applications

1. Portfolio Selection

Setup: n stocks, allocate wealth daily.

Decision: Portfolio p_t (probability distribution over stocks).

Loss: Negative return $-\langle r_t, p_t \rangle$ where r_t is return vector.

Goal: Compete with best fixed portfolio in hindsight.

Algorithm: MWU on stocks achieves $O(\sqrt{T \log n})$ regret.

2. Online Classification

Setup: Predict labels $y_t \in \{-1, +1\}$ for instances x_t.

Experts: Classifiers $h_1, ..., h_n$.

Loss: 0-1 loss or hinge loss.

Result: Learn to combine classifiers with low regret.

3. Game Playing

Setup: Repeated matrix game against adversary.

Decision: Mixed strategy (probability over actions).

Loss: Game payoff.

Result: MWU converges to Nash equilibrium in zero-sum games.

7. Key Pitfalls

Regret vs generalization error: Regret compares to best fixed strategy, not true risk. Good regret doesn't imply low test error.

Learning rate tuning: Optimal η depends on T. If T unknown, use doubling trick or adaptive learning rates.

Oblivious adversary assumption: MWU assumes adversary doesn't see current randomization. Against adaptive adversary, need stronger tools.

$O(\sqrt{T})$ is unavoidable: No algorithm can do better than $O(\sqrt{T})$ in adversarial setting without additional assumptions.

Log factors matter: Dependence on n is $\log n$ (exponentially better than linear). This is why MWU scales to many experts.

Losses must be bounded: Analysis assumes $\ell_{t,i} \in [0, 1]$. For unbounded losses, need different analysis or clipping.